

* Architecture & Overview *

ARM cortex m4 @ 100MHz

FPU: single precision (float)

Flash: 512 kB

SRAM: 128 kB

Display: $240 \times 320 = 76,800 \times 2 \text{ bytes} = 150 \text{ kB}$ X

$120 \times 160 = 19,200 \times 2 \text{ bytes} = 38.4 \text{ kB}$ ✓

SPI: $\sim 42 \text{ MHz} \rightarrow$ full DMA push $\approx 14 \text{ ms}$ ($\sim 71 \text{ fps}$)

SPI TFT 2.4" display is 8 bytes & has dedicated DC (517789)

- 1) Remove legacy files $\Rightarrow$ Create SDL2 based simulator
 - 2) Create board config as header $\Rightarrow$ inject into the engine
 - 3) Tweak & optimize.
-

Assets extractor :-

We need assets which can be found in shareware (only h/1)

Now, we need a script to extract this. Lets first study it:

- We have the VSWAP.WL1 file, as the name suggests it uses 'VSWAP' container. It is just a 'blob' of data with headers & contains a table of contents

It uses 'VSWAP' container. It is just a 'blob' of data with header & serves as table of contents.

- It actually doesn't have RGB values in it, rather each pixel is a byte (0-255) which points to specific colour in VGA panel
- It stores it in column major style so that it is easier for the raycaster to render it.

So the script must have:

- 1.) Read first 6 bytes to find Sprite Start Offset
As the file is walls → textures → sound
 - 2.) 32 bit int. table where each int. is a pointer to a byte location in file where specific texture begins
 - 3.) For each of those N wall indices, we jump & extract 4096 bytes as walls are 64×64
 - 4.) Preserve format i.e. column major, L → R.
-

g-pfnVectors inside .isr_vector (interrupt service routine)

1.) When cortex m4 boots up, it doesn't know what C code is & expects hardcoded layout at 0x08000000;

- We fetch first 4 bytes directly into **Stack Pointer (SP)** set to **-estack**
- Next 4 bytes are **Program Counter (PC)** i.e. -Reset_Handler

In main.c, we define

SCB_VTOR = 0x08000000 which explicitly tells CPU's control block where this table is.

2.) After this we need to copy the .data & zero out .bss section.

- **CopyDataInit(.data)** :- Takes initialized vars like sprite arrays & copies from flash to ram
- **FillZeroBss(.bss)** :- Zeros uninitialized variables. If not done, garbage values may appear.

* Clock Cycle Engine;

This is the heart of the system as the game is tick based

STM32F411CEU6 supports 100MHz but 96 MHz works for us.

We configure the Phase Locked Loop (PLL) register as:-

RCC_PLLCFGR = (8 << 0) | ^(5:0) **PLLM** = 8 (001000) divides by 8
(96 << 6) | ^(14:6) **PLLN** = 96 (00110000) Feedback freq. mult. via phase comparator
(0 << 16) | ^(17:16) **PLLQ** = 0 (00) Splits by 2 i.e. divide by 2
(0 << 22) | ⁽²²⁾ **PLLSRC** = 0 (0) Hardware multiplies link PLL to HST

(0 < 16) | PLLP = 0 (00) Splits by 2 i.e. divide by 2

(0 < 22); PLLSRC = 0 (0) Hardware multipliers, links PLL to HSI

Normally, f_{HSI} based on internal oscillator is 16 MHz

The internal voltage Controlled Oscillator (VCO) requires 1-2 MHz for stability. Then it multiplies (N) & then we scale it down to our requirement using Prescaler (P). We get

$$\begin{aligned} \text{SYS CLK} &= \left(\frac{f_{HSI}}{m} \right) \times \frac{N}{P} \quad ; \quad m = 1/8 \\ & \quad \quad \quad N = 96 \\ & \quad \quad \quad P = 2 \\ &= \underline{\underline{96 \text{ MHz}}} \end{aligned}$$

* Low Latency Flash : As the flash runs at 20 MHz which is < 96 MHz, we need to fix this by slowing down fetch cycles.

FLASH_ACR = (1 < 8) | PRFTEN: Prefetch enables other 128 bit bus to prefetch
(1 < 9) | ICEN: Instruction cache (Ultra Fast Internal SRAM array)
(1 < 10) | DCEN: Data cache (— 11—)
3; Writes lowest 3 = 0011. This reroutes the CPU through a 3 Stage Flip-Flop to induce 3 cycle delay.

* Systick & ms_delay : As humans we normally don't operate based on systick. We have seconds, minutes etc. Based on config so far we have 96 MHz = 96,000,000 resolution of a second. This is required as after every millisecond, the execution pauses & jumps to Systick_Handler & should do something with a counter.

`RCC_CR |= (1 << 24);` Turn on PLL

`while (!(RCC_CR & (1 << 25)));` Wait for lock otherwise erratic freq. appear

`RCC_CFGR = (4 << 10);` ^(12:10) `PRE1 = 4 (100)` low speed peripheral bus prescaler.
(clumps by 2/2 i.e. to 48 MHz for safety.)

`RCC_CFGR1 = 2;`

^(3:0) `SW = 2 (10)` Shifts from HSI to stable PLL

Now, we define

`SYSC_VAL = 0;` We load internal 24 bit down counter

`SYSC_LOAD = 96000-1;` with 95999. At 96MHz this

`SYSC_CTRL = 7;` decrements by 1 per cycle.

Thus it takes 1ms to count till 0.

The math is simple: we have 96,000,000 ^(cycles) resolution of a second. When something of this resolution cycle does 96000 cycles, one set of 96000 cycles will take 0.001s or 1ms!

Now, we have link betⁿ machine & human states. & we keep the least count 1ms. OK! Now something should happen after 1ms? Cascading Sequence!

* Cascading Sequence:- As Cortex M4 has several coprocessors & other asynchronous hardware, the program is not purely sequential. So when SysTick hits 0, it fires the NVIC (Nested Vectored Interrupt Controller). Why? Because:

i) Decouple Engine time from frame rate - Now, we can keep track of time regardless of what's happening in engine/CPU.

ii) Zero-Drop Input Precision (Antighosting) - Ever wondered while playing an online multiplayer, you clicked fire but you got shot instead? This is something similar. When CPU faces some hard math like multiple spits & enemy rendering then

hard math like multiple sprite & enemy rendering then when fire is pressed, it should fire regardless & update frame. This is achieved by this method. After every 1ms, the hardware checks the state of all inputs like joystick/buffer & then resumes.

So how we achieve this?

i.) Physical Context Switching (Pause): CPU can't go GTFU & pause. It needs to remember what it was doing before interrupt. This is done by pushing the state of 8 critical registers onto active stack pointer (-esp):

- Program Counter (PC): So it knows which line of code
- Status Register (xPSR): so it remembers math comparison was true or false
- R0: R3, R12, L R: Why? RTFM!

This automatically done by hardware, if configured correctly!

ii.) Vector fetch & jump: NVIC looks at the vector table address at 0x08000000, locates 16th entry (SysTick vector), reads memory address of SysTick_Handler() & forces PC to jump straight to that instruction

iii.) Execution: C runs ms_ticks++ increments & input_ticks() updates button pins.

iv.) Unstacking (resume): When handler is done, it executes a special return value (EXC_RETURN). The CPU hardware detects this, reverse entire procedure & drops the execution right back to the point where it was paused.

ISR must never have heavy code/calculations. It's a

ISR must never have heavy code / calculations. It is a dot 2121 that strikes quickly, does the job & leaves so other routines can function properly.

Display and rendering

Monday, 25. May 2026

5:52 PM

The display is 320×240 (RGB565)

i.e. 320×240 pixels & 16 bit RGB (RRRR GGGG BBBB) bits.

$$320 \times 240 \times 2 \text{ bytes} = 153,600 \text{ Bytes or } 150 \text{ kB}$$

but our hardware has 128 kB.

so we render the screen at 160×120 paletteized array $\Rightarrow 19,200 \text{ bytes} \approx 18.7 \text{ kB}$

- ② We use SPI DMA to upscale this 2x on the fly.
- ③ The DMA occurs from asynchronous ping pong line buffering architecture

* Peripheral Clock Gating

`RCC - AHB1ENR |= (1 << 0) | (1 << 1);` Enable GPIO A & GPIO B clock

`RCC - APB2ENR |= (1 << 12);` Enable SPI 1 clock

`RCC - AHB1ENR |= (1 << 22);` Enable DMA2 clock

This will turn them ON

* High speed Pin Routing:

`GPIOA - MODER |= ((2 << (5 * 2)) | (2 << (7 * 2)));` (11:10, 15:14) = 10
Config PA5, PA7 for alternate mode

`GPIOA - AFRL |= ((0 * 5 << (5 * 4)) | (0 * 5 << (7 * 4)));` (23:20, 31:28 = 0101)
Commit PA5 & PA7 to line AF5 (SPI1)

$\text{GPIOA_OSPEEDR} |= (3 \ll (3*2)) |$ Sets PA3, PA4, PA5, PA7
 $(3 \ll (4*2)) |$ to very high speed mode
 $(3 \ll (5*2)) |$
 $(3 \ll (7*2));$

* SPI Bus optimisation :-

$\text{SPI1_CR1} = (1 \ll 2) |$ $\text{MSTR} = 1$, SPI1 operates as master controller
 $(1 \ll 9) |$ $\text{SSM} = 2$ } Enable Software slave management
 $(1 \ll 8) |$ $\text{SSI} = 1$ } Avoids NSS pin & enables bit bashing
 $(0 \ll 3);$ $(5:3) \text{BR} = 0$, Sets baud rate prescaler to divide by 2 so 96 → 48 MHz

* DMA Stream3 blueprint :-

$\text{DMA2_S3PAR} = (\text{uint32_t}) \& \text{SPI1_DR};$

Set peripheral Address Register. Points the target destination of Channel 3 directly to the physical memory location of SPI1 Data Register.

$\text{DMA2_S3CR} =$ $(3 \ll 25) |$ $(27:25) \text{CHSEL} = 3$, Maps this DMA stream to channel 3 which is hardwired to chips internal crossbar matrix
 $(1 \ll 16) |$ $(17:16) \text{PL} = 01$, Medium priority
 $(1 \ll 13) |$ $(14:13) \text{MSIZE} = 01$, Config data width to 16 bit half words. Every transfer will grab 2 bytes simultaneously from column any
 $(1 \ll 11) |$ $(12:11) \text{PSIZE} = 01$
 $(1 \ll 10) |$ $(10) \text{MINC} = 1$; Memory Increment Enable, auto increment ptr by 2 byte address
 $(1 \ll 6) |$ $(7:6) \text{DIR} = 01$; Directionality matrix as memory to peripheral

* Async Ping-Pong Arch :-

We need macros to set SPI at 8 bit & 16 bit as our colour calculations are shift capable. Some devices have a 16-bit

We need macros to set SPI at 8bit & 16 bit as our colour calculations are 8bit while screen driver is 16bit.

SPI-set 8bit();

spi - tx - cmd(0x2A); *ASET: Col.AddressSet to define Horiz. boundaries*

spi - tx - data(0);

- 1 - data(0);

- 1 - data(319 >> 8);

- 1 - data(319 & 0xFF)

send 16bit col. split into 2 8bit bytes followed by 16bit end col(319) split into two bytes (319 >> 8) high byte, (319 & 0xFF) master low byte. This restricts internal pointer from 0 to 319.

DO SAME FOR ROW

spi - tx - cmd(0x2C) *Display controller gets ready for data stream*

DC-DATA(); *DC ON*

cs-LOW(); *CS ON*

set-spi-16bit(); *set to 16 bit.*

int A_buf = 0; *Ping & Pong state var*

convert_row-2x(); *Scale 2x current row(CPU Task)*

for (ENTIRE ROWS/SCREEN-HEIGHT)

int B_buf = A_buf ^ 1; *XOR switch*

push-dma(A_buf); *(DMA Task)*

convert_row-2x(); *(CPU Task)*

wait-dma();

push-dma(A_buf); *cheap trick for 2x verticle render*

wait-dma();

A_buf = B_buf;

* 2x upscaler :

for (SCREEN-WIDTH)

pixel = map_8bit_16bit ();

buf[i*2] = pixel;

buf[i*2+1] = pixel;

} Actual horizontal stretch

* 8 bit to 16 bit,

for (256) as its 8 bit

uint8r = wolf_vga[i][0]

g = -1- [i][1]

b = -1- [i][2]

wolf_vga is palette
data extracted from
VSwap.

uint_16 palette_16bit [i] = [RRRRR bits! GGGGG BBBB]

= ((r >> 3) << 11) | Take red & shift right 3 bits, then shift left 11 bits.

((g >> 2) << 5) | Take g, shift right 2 bits, then shift left 5 bits.

((b >> 3); Take b, shift right 3 bits.

How this works? Lets consider actual index from palette

wolf_rgb[6][0] = 170 (10101010)

[6][1] = 85 (01010101)

[6][2] = 0 (00000000)

so, (r >> 3) = 10101010 >> 3 = 00010101

<< 11 = 1010100000000000 (11 bits for g & b)

((g >> 2) = 01010101 >> 2 = 00010101

<< 5 = 0000001010100000

((b >> 3) = (same effect as r)

combining :- 1010101010100000

Now CPU needs to locate array index slot [6]

As it uses 2 bytes, it can jump addresses as

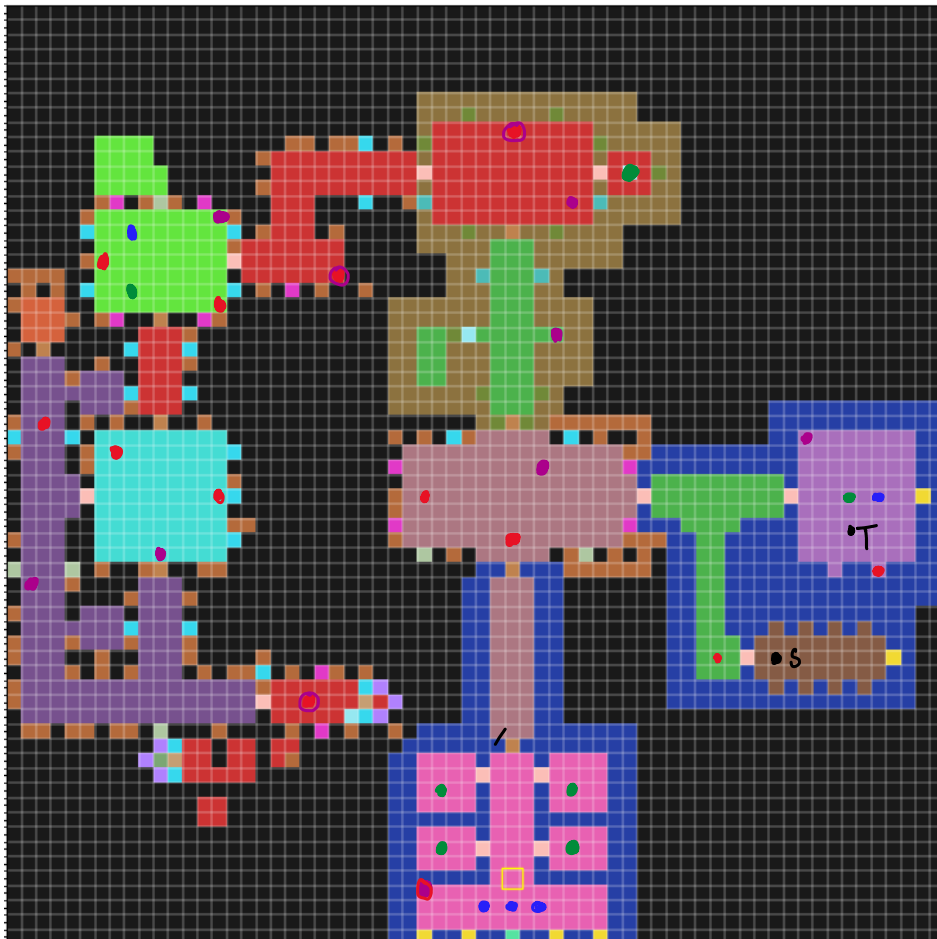
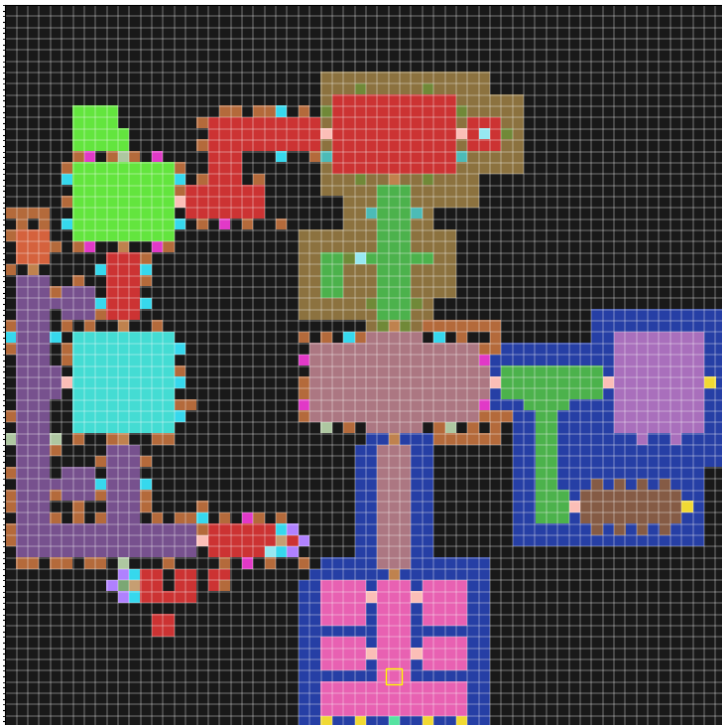
$$\begin{aligned}\text{Adr-jump} &= \text{Base-adr-idx} \times \text{2 bytes-per-element} \\ &= 6 \times 2 = 12 \text{ bytes}\end{aligned}$$

If palette array sits at $0x200001000$ then for idx[6]

it will grab 16 bits starting from $0x200001000 + 12 = 0x20000100c$

Level Design

Monday, May 18, 2026 9:29 PM



S: spawn = (54, 44)

T: Testroom = (57.5, 37.5)

- Enemy 1 • Health
- Enemy 2 • Misc
- Ammo

D. spawn (54, 14)

T: Testroom = (57.5, 37.5)

• Enemy 2
• Ammo
• Misc

(59, 38) (22, 18) (14, 33)

(34, 36) (14, 20) (2, 28)

(28, 33) (6, 17) (20, 47)

(34, 8) (7, 30) (40, 60)

(59, 33) (8, 15)

(32, 61)

(34, 61)

(36, 61)

(54, 29) (14, 14)

(36, 31) (10, 37)

(28, 60) (1, 39)

(38, 13) (36, 22)

(57, 33) (38, 53)

(29, 53) (42, 11)

(29, 57) (8, 19)

(38, 57)

(51, 44) - Dead body

(59, 31) - Table

(61, 37) - Barrel

(33, 29) } Pot

(35, 21)

(28, 31) - Lamp

(33, 16)

(35, 16) } Barrel

(39, 10) } Pot

(39, 12)

(29, 10) } Lamp

(29, 12)

(7, 15) - Table

(1, 48) - Lamp

(8, 35) - Table

(14, 24) - Lamp

(6, 32) } Barrel

(6, 34) }

(9, 37) } Pot

(11, 37) }